

Керівництво з принципів проектування програмного забезпечення

1. DRY (Don't Repeat Yourself) - Не повторюй себе

Принцип, який стверджує, що кожен елемент знань повинен мати єдине, недвозначне представлення в системі.

```
// Поганий приклад (дублювання коду)
function calculateRectangleArea(width: number, height: number): number {
    return width * height;
}

function calculateRectanglePerimeter(width: number, height: number): number {
    return 2 * (width * height);
}

// Гарний приклад (усунення дублювання)
class Rectangle {
    constructor(private width: number, private height: number) {}

    calculateArea(): number {
        return this.width * this.height;
    }

    calculatePerimeter(): number {
        return 2 * (this.width + this.height);
    }
}
```

2. KISS (Keep It Simple, Stupid) - Роби простіше

Принцип, що закликає уникати зайвих ускладнень і писати максимально простий код.

```
// Поганий приклад (ускладнене рішення)
function isPrime(num: number): boolean {
    if (num <= 1) return false;
    for (let i = 2; i < Math.sqrt(num); i++) {
        if (num % i === 0) {
            return false;
        }
    }
    return true;
}

// Гарний приклад (просте та зрозуміле рішення)
```

```
function isPrime(num: number): boolean {
    return num > 1 &&
        Array.from({length: num - 2}, (_, i) => i + 2)
            .every(divisor => num % divisor !== 0);
}
```

3. YAGNI (You Aren't Gonna Need It) - Вам це не знадобиться

Принцип екстремального програмування, який радить не додавати функціональність, доки вона не стає необхідною.

```
// Поганий приклад (передчасна оптимізація)
class UserService {
    private users: User[] = [];
    private cache: Map<string, User> = new Map();

    // Кешування, яке може і не знадобитися
    getUserId(id: string): User | undefined {
        if (this.cache.has(id)) {
            return this.cache.get(id);
        }
        const user = this.users.find(u => u.id === id);
        if (user) {
            this.cache.set(id, user);
        }
        return user;
    }
}

// Гарний приклад (мінімальна реалізація)
class UserService {
    private users: User[] = [];

    getUserId(id: string): User | undefined {
        return this.users.find(u => u.id === id);
    }
}
```

4. SOLID - Принципи об'єктно-орієнтованого проектування

(Короткий опис, детальніше в попередньому керівництві)

- **S**ingle Responsibility Principle
- **O**pen/Closed Principle
- **L**iskov Substitution Principle
- **I**nterface Segregation Principle
- **D**ependency Inversion Principle

5. GRASP (General Responsibility Assignment Software Patterns)

Набір принципів для призначення відповідальностей класам та об'єктам.

```
// Приклад застосування GRASP - принцип Creator
class Order {
    private items: OrderItem[] = [];

    addItem(item: OrderItem) {
        this.items.push(item);
    }

    calculateTotal(): number {
        return this.items.reduce((total, item) => total + item.price, 0);
    }

    // Клас Order відповідає за створення пов'язаних з ним об'єктів
    createInvoice(): Invoice {
        return new Invoice(this);
    }
}
```

6. Law of Demeter (Принцип Деметри)

Принцип, що обмежує взаємодію об'єктів, щоб зменшити зв'язаність.

```
// Поганий приклад (порушення принципу Деметри)
class User {
    getWallet(): Wallet {
        return this.wallet;
    }
}

class PaymentService {
    processPayment(user: User, amount: number) {
        // Забагато знань про внутрішню структуру User
        const balance = user.getWallet().getBalance();
        user.getWallet().withdraw(amount);
    }
}

// Гарний приклад
class User {
    private wallet: Wallet;

    withdrawFromWallet(amount: number): boolean {
        return this.wallet.withdraw(amount);
    }
}
```

```
class PaymentService {
  processPayment(user: User, amount: number) {
    // Мінімум залежностей
    user.withdrawFromWallet(amount);
  }
}
```

7. Separation of Concerns (Розділення відповідальностей)

Принцип розділення програми на окремі, слабо пов'язані частини.

```
// Розділення логіки завантаження, обробки та відображення даних
class DataLoader {
  async fetchData(url: string) {
    const response = await fetch(url);
    return response.json();
  }
}

class DataProcessor {
  processData(data: any[]) {
    return data.filter(item => item.isValid);
  }
}

class DataRenderer {
  render(data: any[]) {
    // Логіка візуалізації даних
  }
}
```

Висновок

Ці принципи допомагають:

- Створювати більш чистий та зрозумілий код
- Зменшувати складність програмних систем
- Підвищувати гнучкість та підтримуваність коду
- Прискорювати процес розробки

Застосування цих принципів вимагає практики та розуміння контексту. Не варто сприймати їх як догму - важливо використовувати їх розумно та доречно.

Керівництво допоможе вам покращити якість вашого програмного забезпечення!